

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software

Aplicaciones Distribuidas (ISR-701) — Unidad 3: Bases de Datos Distribuidas

Informe Técnico Individual

Análisis de Consistencia y Replicación en Bases de Datos Distribuidas

Caso de estudio: Microservicio **sga-soporte** — Módulo de Gestión de Tickets

Código: TA-IND-02

Estudiante: Juliana Romina Emanuel Pino

Equipo PFC: ACC – Soporte Técnico ISP (Gestión de Tickets)

Asignatura: Aplicaciones Distribuidas (ISR-701)

Unidad: 3 – Bases de Datos Distribuidas

Docente: Prof. Guerrero Ulloa G.C.

Fecha: 14 de julio de 2026

Resumen

Este informe analiza el modelo de consistencia y la estrategia de replicación realmente implementados en el subsistema de soporte técnico (**sga-soporte**) del equipo ACC, que expone la gestión de tickets del Sistema de Gestión Académica de la UTEQ. El sistema no replica datos entre nodos propios: dos microservicios independientes (**sga-soporte** y **sga-principal**) comparten una única instancia primaria de PostgreSQL alojada en Supabase, diferenciada por esquemas. Se caracteriza este diseño como un modelo de *consistencia fuerte por concentración en un solo nodo autoritativo*, se contrasta con los marcos CAP y PACELC, y se evalúan dos alternativas — réplicas de lectura asíncronas (patrón líder-seguidor) y base de datos por servicio con propagación por eventos — mediante una tabla comparativa de cuatro criterios técnicos. La discusión crítica concluye que el modelo actual es adecuado para el volumen y la criticidad actuales del dominio de tickets, pero que compromete la disponibilidad y la evolución independiente de los servicios, por lo que se proponen mitigaciones concretas sin abandonar PostgreSQL como motor principal.

Palabras clave: consistencia, replicación, CAP, PACELC, microservicios, PostgreSQL, base de datos compartida.

Contents

1	Introducción	3
2	Marco teórico	3
2.1	Modelos de consistencia	3
2.2	Teoremas de compromiso: CAP y PACELC	4
2.3	Estrategias de replicación	4
3	Análisis del modelo de consistencia del PFC	5
3.1	Caracterización del modelo implementado	5
3.2	Justificación técnica ligada al dominio	6
4	Evaluación de alternativas	7
4.1	Alternativa A: réplicas de lectura asíncronas (líder-seguidor)	7
4.2	Alternativa B: base de datos por servicio con propagación por eventos	8
5	Discusión crítica	8
6	Conclusiones	9
7	Declaración de uso de IA generativa	10

1. Introducción

El Sistema de Gestión Académica (SGA) de la Universidad Técnica Estatal de Quevedo se construye como un conjunto de microservicios independientes que colaboran mediante autenticación JWT compartida. Dentro de este ecosistema, el equipo ACC es responsable del subsistema de **Soporte Técnico**, materializado en el microservicio **sga-soporte** (puerto 8081, backend Spring Boot, frontend React/Vite/Tailwind en el puerto 5173) y su contraparte **sga-principal** (puerto 8080), que gestiona usuarios y emite los tokens de autenticación que **sga-soporte** valida localmente.

El dominio funcional de **sga-soporte** es la gestión de tickets de incidencias: creación, asignación, cambio de estado (**ABIERTO**, **EN_PROCESO**, **RESUELTO**, **CERRADO**), comentarios asociados — incluyendo notas internas no visibles para el usuario final — y estadísticas agregadas por estado. Se trata de un dominio con requisitos de integridad moderados: un ticket debe tener en todo momento un estado único y consistente, visible de forma idéntica para el usuario que lo creó, el técnico asignado y cualquier panel de auditoría, pero no exige la latencia de submilisegundo ni el volumen transaccional de un sistema financiero o de reservas de alta concurrencia.

Este informe responde a la pregunta orientadora del TA-IND-02: *“es el modelo de consistencia elegido el óptimo para el dominio del sistema?”* Para ello, la Sección 2 revisa el marco teórico de modelos de consistencia, los teoremas CAP y PACELC, y las estrategias de replicación. La Sección 3 caracteriza el modelo realmente implementado en **sga-soporte**, a partir de la inspección directa del código fuente (capas **entity**, **repository**, **service** y **application.properties**). La Sección 4 evalúa dos alternativas de consistencia/replicación frente al modelo actual. La Sección 5 discute críticamente si el modelo elegido es óptimo, y la Sección 6 concluye con recomendaciones concretas.

2. Marco teórico

2.1 Modelos de consistencia

La consistencia en un almacén de datos distribuido no es una propiedad binaria (“consistente” o “inconsistente”), sino un espectro ordenado de garantías con semántica formal precisa [6]. En orden decreciente de fuerza:

- **Consistencia fuerte / linealizabilidad:** toda operación de lectura observa el efecto de la última escritura confirmada, como si existiera una única copia de los datos y un orden global total de operaciones [6]. Es la garantía más costosa de sostener porque exige coordinación entre réplicas antes de confirmar cada escritura.
- **Consistencia secuencial:** todas las réplicas observan las operaciones en el mismo orden, pero ese orden no tiene por qué coincidir con el orden real en el tiempo [6].
- **Consistencia causal:** solo se garantiza el orden entre operaciones causalmente relacionadas; operaciones concurrentes pueden observarse en órdenes distintos en distintos nodos [6].
- **Lectura de los propios cambios (read-your-writes):** un cliente siempre ve sus propias escrituras previas, aunque no necesariamente las de otros clientes de forma inmediata [6].

- **Consistencia eventual:** en ausencia de nuevas escrituras, todas las réplicas convergen finalmente al mismo estado, sin garantía de cuándo ocurre esa convergencia [6].

Cuanto más fuerte es el modelo, mayor es la coordinación (y por tanto la latencia y el acoplamiento entre nodos) necesaria para sostenerlo; cuanto más débil, mayor es la disponibilidad y menor la latencia percibida, al costo de exponer estados transitoriamente divergentes entre réplicas [6].

2.2 Teoremas de compromiso: CAP y PACELC

El teorema CAP, formalizado por Gilbert y Lynch a partir de la conjetura de Brewer, establece que ante una partición de red un sistema distribuido no puede garantizar simultáneamente consistencia (*Consistency*) y disponibilidad (*Availability*); debe sacrificar una de las dos mientras la partición persiste, conservando siempre tolerancia a particiones (*Partition tolerance*) [4].

CAP, sin embargo, solo describe el comportamiento del sistema *durante* una partición, que es un evento relativamente infrecuente. Abadi extiende el análisis con la formulación PACELC: si hay Partición (P), el sistema elige entre Availability (A) y Consistency (C), *pero además (Else)*, en operación normal sin particiones, todo sistema replicado enfrenta un segundo compromiso entre Latencia (L) y Consistencia (C) [1]. Un sistema que replica de forma síncrona para garantizar consistencia fuerte paga esa consistencia con latencia adicional en cada escritura, incluso cuando la red funciona con normalidad; un sistema que replica de forma asíncrona reduce la latencia percibida pero admite una ventana de posible pérdida o desactualización de datos. PACELC es el marco más adecuado para analizar sistemas que, como el del presente informe, no operan sobre múltiples nodos de datos propios y por tanto rara vez enfrentan particiones internas, pero sí toman decisiones cotidianas de diseño que afectan el eje latencia-consistencia.

2.3 Estrategias de replicación

Sabaghian, Khamforoosh y Ghaderzadeh, en su revisión del estado del arte sobre replicación y ubicación de datos en sistemas distribuidos, caracterizan las estrategias de replicación según dónde reside la autoridad de escritura y cómo se propaga el cambio [5]:

- **Primario-respaldo (líder-seguidor):** un único nodo líder acepta escrituras y las propaga a nodos seguidores de solo lectura; simplifica la resolución de conflictos porque nunca hay dos escritores concurrentes sobre el mismo dato [5].
- **Multi-líder:** varios nodos aceptan escrituras de forma independiente y sincronizan cambios entre sí, lo que exige mecanismos de resolución de conflictos (por ejemplo, resolución por marca temporal o CRDTs) [5].
- **Sin líder por quorum:** cualquier réplica acepta lecturas y escrituras, y la consistencia se negocia por solapamiento de quorum ($W + R > N$) en cada operación, como en Dynamo [3].
- **Replicación síncrona frente a asíncrona:** la replicación síncrona confirma la escritura solo tras propagarla a las réplicas requeridas (mayor consistencia, mayor latencia); la asíncrona confirma de inmediato en el nodo local y propaga en segundo plano (menor latencia, ventana de inconsistencia) [5].

Los sistemas de referencia industrial ilustran los extremos de este espectro. Dynamo, el almacén clave-valor de Amazon, prioriza la disponibilidad mediante replicación sin líder por quorum y consistencia eventual, aceptando que distintas réplicas puedan divergir temporalmente a cambio de nunca rechazar una escritura [3]. En el extremo opuesto, Spanner, de Google, ofrece consistencia externa — transacciones globalmente consistentes incluso a través de centros de datos distantes — apoyándose en sincronización temporal de alta precisión (TrueTime) para ordenar transacciones sin necesidad de comunicación adicional entre réplicas [2]. Estos dos casos definen los polos frente a los cuales se puntúa, en la Sección 4, el modelo del sistema propio.

Cabe señalar que la coordinación de datos entre microservicios no se limita a la replicación clásica entre nodos de una misma base de datos. El estudio empírico de Laigner *et al.*, basado en la revisión de aplicaciones de microservicios de código abierto y una encuesta a más de 120 profesionales, clasifica la persistencia de microservicios en tres patrones: tablas privadas que comparten servidor y esquema de base de datos, esquema por microservicio sobre un servidor común, y servidor de base de datos independiente por microservicio; y encuentra que la mayoría de los profesionales prefiere encapsular el estado de cada microservicio en su propio servidor, precisamente para lograr acoplamiento débil, aislamiento de fallos y evolución independiente del esquema [8]. La gestión de la consistencia de datos entre servicios sigue siendo, de hecho, uno de los retos de diseño más señalados en la literatura reciente sobre arquitecturas de microservicios [7].

3. Análisis del modelo de consistencia del PFC

3.1 Caracterización del modelo implementado

La inspección del código fuente de `sga-soporte` permite caracterizar el modelo de datos con precisión, sin necesidad de inferencias. El archivo `application.properties` declara una única fuente de datos:

```
spring.datasource.url=jdbc:postgresql://aws-1-us-east-1.pooler.supabase.com:5432/postgres
spring.jpa.properties.hibernate.default_schema=sga_principal
spring.datasource.hikari.maximum-pool-size=2
```

Es decir: `sga-soporte` no posee una base de datos propia. Se conecta, a través de un *pooler* de conexiones, a la **misma instancia física de PostgreSQL gestionada por Supabase** que utiliza `sga-principal`, diferenciándose únicamente por esquema: las entidades `TicketSoporte` y `ComentarioTicket` se mapean explícitamente al esquema `soporte` (`@Table(schema = "soporte")`), mientras que el esquema por defecto de Hibernate para el resto de operaciones es `sga_principal`, el esquema propio del microservicio de usuarios. No existe bus de mensajes, cola de eventos ni mecanismo de sincronización entre nodos de datos: la única autoridad de escritura es el propio motor de PostgreSQL de Supabase.

Esto tiene una consecuencia conceptual importante para el informe: **no hay, en sentido estricto, replicación de datos diseñada por el equipo**. Lo que existe es un *modelo de nodo único compartido entre servicios*, apoyado en las garantías ACID nativas de un motor relacional. La Figura 1 ilustra esta topología.

Como se observa en la Figura 1, la única forma de acoplamiento entre `sga-soporte` y `sga-principal` a nivel de datos es indirecta: ambos escriben en la misma instancia, pero no existe una tabla o entidad compartida a nivel de dominio de tickets (los usuarios se resuelven por `username`, un identificador

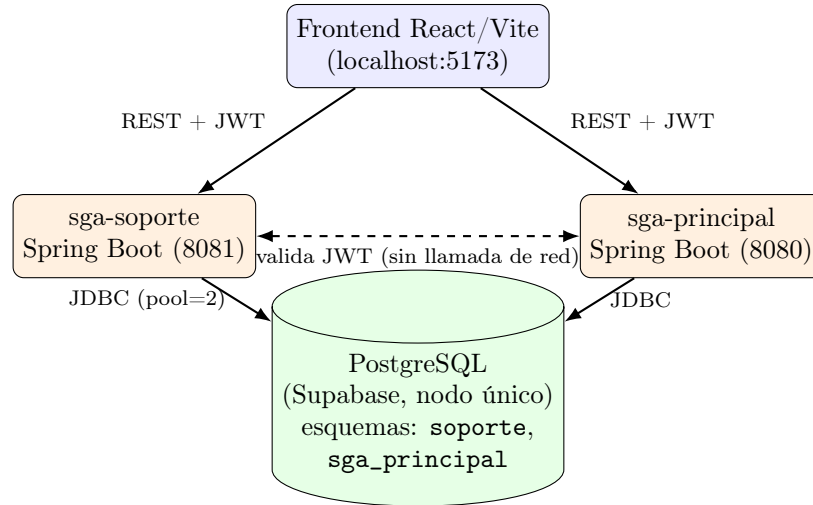


Figure 1: Topología de datos de **sga-soporte**: dos microservicios independientes comparten una única instancia primaria de PostgreSQL, diferenciados por esquema. No existe replicación de datos propia del equipo; la consistencia entre servicios depende enteramente del nodo relacional compartido.

de texto embebido en el JWT y en los campos `creadoPor/asignadoA`, no por clave foránea de base de datos). El acoplamiento de autenticación es a nivel de aplicación (`jwt.secret` idéntico en ambos servicios), no de base de datos.

A nivel de operaciones, la capa `TicketService` usa `@Transactional` de Spring en cada método de escritura (`crear`, `actualizar`, `cambiarEstado`) y `@Transactional(readOnly = true)` en las de lectura. Al no existir más que un nodo de datos, cada transacción se resuelve con las garantías ACID por defecto de PostgreSQL (nivel de aislamiento *read committed* salvo configuración explícita), sin necesidad de protocolos de confirmación distribuida (2PC, sagas, etc.). El tamaño de pool de conexiones, limitado a dos (`maximum-pool-size=2`), es una restricción de la capa gratuita de Supabase y no una decisión de consistencia, pero condiciona directamente la disponibilidad percibida bajo concurrencia, como se discute en la Sección 5.

3.2 Justificación técnica ligada al dominio

Leído bajo CAP, el sistema no enfrenta el dilema clásico de forma directa: al no tener réplicas propias de datos, no existe partición *interna* que obligue a elegir entre consistencia y disponibilidad del lado del equipo ACC. La partición relevante en este diseño es externa: la conectividad entre los microservicios (desplegados, por ejemplo, en distintos contenedores o máquinas) y la instancia de Supabase, que corre en la infraestructura de un proveedor gestionado. Si esa conectividad falla, **sga-soporte** pierde disponibilidad por completo (no puede leer ni escribir), en vez de degradar a un modo de disponibilidad parcial con datos potencialmente obsoletos. Es, en la terminología de CAP, un sistema que **renuncia a la disponibilidad ante una partición, en favor de la consistencia**, aunque esa renuncia no es una decisión de diseño explícita del equipo, sino una consecuencia de no tener réplicas a las que degradar.

Leído bajo PACELC, el eje relevante es el segundo (*else*: latencia contra consistencia en operación normal). Como no hay replicación, no existe el compromiso latencia-consistencia que aparece en

sistemas con múltiples réplicas: cada lectura obtiene automáticamente el dato más reciente, porque solo hay un lugar donde el dato puede estar. El costo de esta simplicidad se paga en otro eje, no capturado directamente por PACELC pero sí relevante para el dominio: la **latencia de red hacia un único proveedor externo** (Supabase, alojado en `us-east-1`) y el **límite de concurrencia** impuesto por el pool de dos conexiones, que puede convertirse en un cuello de botella si varios técnicos consultan o actualizan tickets simultáneamente.

Para el dominio de gestión de tickets de soporte técnico, esta elección es razonable en varios sentidos:

1. El volumen de escrituras es bajo (creación y actualización de tickets por un número acotado de técnicos y usuarios de una institución universitaria), por lo que la ausencia de réplicas de escritura no representa hoy un cuello de botella crítico.
2. La integridad del estado de un ticket es más valiosa que la disponibilidad extrema: que dos técnicos vean el mismo ticket como **ABIERTO** cuando en realidad ya fue **RESUELTO** podría derivar en trabajo duplicado o en información contradictoria para el usuario que reporta el problema. El modelo de nodo único, al no admitir estados divergentes entre réplicas, elimina por construcción esa clase de error.
3. El acoplamiento de autenticación por JWT simétrico evita que la validación de identidad dependa de una llamada de red a `sga-principal` en cada petición, lo que sí habría introducido una dependencia explícita de disponibilidad entre servicios.

No obstante, esta misma elección genera los riesgos que se puntualizan en la Sección 5: dependencia total de un proveedor externo, imposibilidad de escalar lecturas horizontalmente, y un acoplamiento de esquema entre dos microservicios que, arquitectónicamente, deberían evolucionar de forma independiente.

4. Evaluación de alternativas

Se contrastan a continuación dos alternativas de consistencia/replicación frente al modelo actual (nodo único compartido), seleccionadas por representar los dos caminos de evolución más plausibles para este sistema: una que mantiene PostgreSQL pero introduce replicación, y otra que separa la base de datos por servicio.

4.1 Alternativa A: réplicas de lectura asíncronas (líder–seguidor)

Consiste en mantener PostgreSQL como líder único de escritura, pero añadir una o más réplicas de solo lectura (*read replicas*) mediante replicación de flujo (*streaming replication*) nativa de PostgreSQL, redirigiendo hacia ellas las consultas de solo lectura de `TicketService` (`listarTodos`, `buscar`, `estadísticas`) [5]. El líder sigue siendo el único punto de escritura, por lo que la consistencia de las escrituras no cambia; lo que cambia es que las lecturas pueden observar un estado ligeramente rezagado (consistencia de lectura de los propios cambios no garantizada por defecto, salvo que se fuerce la lectura al líder tras una escritura reciente).

4.2 Alternativa B: base de datos por servicio con propagación por eventos

Consiste en dotar a **sga-soporte** de su propio esquema físicamente independiente (o incluso de un motor distinto), eliminando la dependencia directa sobre el esquema **sga_principal**, y sincronizar los datos de usuario necesarios (por ejemplo, nombre visible del técnico asignado) mediante eventos asíncronos publicados por **sga_principal** [8]. Este es el patrón *database-per-service*, que exige aceptar consistencia eventual entre servicios a cambio de desacoplarlos por completo.

Table 1: Comparación del modelo actual frente a dos alternativas, según cuatro criterios técnicos.

Criterio	Actual: nodo único	Alt. A: réplicas lectura	Alt. B: BD por servicio
Consistencia	Fuerte (automática)	Fuerte en escritura; eventual en lectura	Eventual entre servicios (propagación por eventos)
Disponibilidad	Baja ante caída del único nodo	Media: lecturas resisten caída del líder	Alta: cada servicio opera aunque el otro falle
Latencia	Baja en escritura; sujeta a pool de 2 conexiones	Baja en lectura (nodo cercano); sin cambio en escritura	Baja en ambos, salvo retraso de sincronización de eventos
Tolerancia a particiones	Nula (un solo punto de fallo)	Parcial (líder sigue siendo punto único)	Alta (fallos aislados por servicio)
Complejidad de implementación	Mínima (sin cambios de infraestructura)	Media (configurar réplicas, enrutar queries)	Alta (esquema propio, bus de eventos, DTOs)

Como resume la Tabla 1, ambas alternativas mejoran la disponibilidad y, en el caso de la Alternativa B, la tolerancia a particiones y la autonomía de despliegue, pero lo hacen a costa de introducir alguna forma de consistencia no estrictamente fuerte y de aumentar la complejidad operativa. Ninguna alternativa es estrictamente superior: la Alternativa A es una mejora incremental de bajo riesgo; la Alternativa B es un rediseño arquitectónico mayor, más alineado con las buenas prácticas de microservicios pero desproporcionado para el volumen de datos actual del módulo de soporte.

5. Discusión crítica

¿Es el modelo de consistencia elegido el óptimo para el dominio del sistema? La respuesta de este informe es **matizada: es adecuado para el estado actual del proyecto, pero no es una elección de diseño deliberada y presenta limitaciones que conviene mitigar antes de escalar.**

A favor del modelo actual: la consistencia fuerte automática que ofrece un único nodo relacional es exactamente la garantía que el dominio de tickets necesita (un ticket no puede estar simultáneamente **ABIERTO** y **RESUELTO** para observadores distintos), y se obtiene sin ningún código adicional de coordinación, lo cual es razonable para un equipo estudiantil con recursos y tiempo limitados y un volumen de datos institucional, no masivo.

En contra, y en línea con lo que la literatura empírica de arquitectura de microservicios describe como una desviación del patrón recomendado de base de datos por servicio [8]: dos servicios que deberían ser independientes (**sga-soporte** y **sga_principal**) comparten físicamente la misma base

de datos, diferenciándose solo por esquema. Esto crea tres riesgos concretos, ninguno de ellos hipotético dado el código revisado:

1. **Punto único de fallo total:** si la instancia de Supabase se degrada o excede su cuota (la propia configuración documenta que se trata de un plan gratuito con pool limitado a dos conexiones), *ambos* microservicios pierden disponibilidad simultáneamente, incluso si uno de ellos no tiene relación funcional con el problema que originó la degradación.
2. **Acoplamiento de esquema oculto:** aunque no hay claves foráneas de base de datos entre `soporte` y `sga_principal`, un cambio futuro en las convenciones de `username` del microservicio principal (por ejemplo, migrar de nombre de usuario a identificador numérico) rompería silenciosamente los campos `creadoPor/asignadoA` de `sga-soporte`, sin que exista un contrato de API explícito que lo prevenga.
3. **Límite de concurrencia artificial:** un pool de dos conexiones JDBC por microservicio, multiplicado por dos microservicios sobre la misma instancia, deja un margen muy estrecho antes de agotar las conexiones disponibles del plan gratuito de Supabase bajo uso concurrente real, lo que en la práctica limita la disponibilidad mucho antes de que el volumen de datos lo justifique.

Estas limitaciones no invalidan la elección actual para la fase de desarrollo en curso, pero sí marcan un techo claro de crecimiento. Como mitigaciones concretas, sin requerir el rediseño completo de la Alternativa B, se recomienda: (i) elevar el plan de Supabase o migrar a una instancia dedicada antes de un despliegue en producción real, ampliando el pool de conexiones; (ii) introducir al menos la Alternativa A (réplica de lectura) para las consultas de estadísticas y listados, que son las de mayor frecuencia y menor sensibilidad a una consistencia estrictamente fuerte; y (iii) formalizar el contrato entre `sga-soporte` y `sga-principal` mediante un DTO de usuario versionado, aun manteniendo la base de datos compartida, para reducir el acoplamiento oculto señalado en el punto 2.

6. Conclusiones

1. El microservicio `sga-soporte` no implementa replicación de datos propia; su modelo de consistencia real es el de un *nodo relacional único compartido* entre dos microservicios, diferenciado por esquema, apoyado en las garantías ACID nativas de PostgreSQL.
2. Leído bajo CAP, el sistema sacrifica disponibilidad ante una eventual partición hacia el proveedor externo, a cambio de consistencia fuerte automática; leído bajo PACELC, evita por completo el compromiso latencia-consistencia propio de los sistemas replicados, trasladando el costo a la latencia de red hacia un único proveedor y al límite de concurrencia del pool de conexiones.
3. Para el volumen y la criticidad actuales del dominio de tickets de soporte técnico, el modelo es funcionalmente adecuado, pero no es resultado de una decisión de arquitectura deliberada, sino de la simplicidad de conectar ambos servicios a la misma instancia gestionada.
4. Las alternativas evaluadas — réplicas de lectura asíncronas y base de datos por servicio con eventos — mejoran disponibilidad y tolerancia a particiones a costa de consistencia estrictamente fuerte y de mayor complejidad operativa; ninguna es urgente hoy, pero la Alternativa A debería priorizarse si el volumen de consultas crece.

5. Se recomienda documentar explícitamente esta decisión arquitectónica en el proyecto (actualmente implícita en `application.properties`) y aplicar las tres mitigaciones propuestas en la Sección 5 antes de cualquier despliegue con usuarios reales de la universidad.

7. Declaración de uso de IA generativa

Para la elaboración de este informe se utilizó el asistente de IA generativa **Claude (Anthropic)** con los siguientes propósitos y alcances:

- **Inspección técnica del código fuente** del repositorio `sga-soporte` (entidades, repositorios, capa de servicio y `application.properties`) para extraer los hechos de arquitectura descritos en la Sección 3 (topología de base de datos, esquemas, transaccionalidad, configuración del pool de conexiones).
- **Búsqueda y verificación de referencias académicas** adicionales a las sugeridas en la guía de la actividad, relacionadas con el patrón de base de datos compartida en microservicios (Sección 2.3, referencias [7] y [8]).
- **Redacción asistida** de las secciones 1 a 6 (introducción, marco teórico, análisis, evaluación de alternativas, discusión crítica y conclusiones), a partir de los hallazgos técnicos verificados por la estudiante sobre su propio proyecto.
- **Generación del diagrama de topología** (Figura 1) en TikZ y de la tabla comparativa (Tabla 1) en formato `booktabs`, a partir de la arquitectura real verificada.
- **Compilación y verificación** del documento `.tex` con `pdflatex` para confirmar ausencia de errores de compilación.

La postura crítica sobre si el modelo es óptimo (Sección 5), la selección del dominio propio de análisis y la revisión final del contenido son responsabilidad de la estudiante, quien validó cada afirmación técnica contra el código fuente real del proyecto antes de la entrega.

References

- [1] D. J. Abadi, “Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,” *Computer*, vol. 45, no. 2, pp. 37–42, 2012. doi: 10.1109/MC.2012.33
- [2] J. C. Corbett *et al.*, “Spanner: Google’s Globally Distributed Database,” *ACM Transactions on Computer Systems*, vol. 31, no. 3, pp. 1–22, 2013, art. no. 8. doi: 10.1145/2491245
- [3] G. DeCandia *et al.*, “Dynamo: Amazon’s Highly Available Key-value Store,” in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP ’07)*, 2007, pp. 205–220. doi: 10.1145/1294261.1294281
- [4] S. Gilbert and N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002. doi: 10.1145/564585.564601
- [5] K. Sabaghian, K. Khamforoosh, and A. Ghaderzadeh, “Data Replication and Placement Strategies in Distributed Systems: A State of the Art Survey,” *Wireless Personal Communications*, vol. 129, pp. 2419–2453, 2023. doi: <https://doi.org/10.1007/s11277-023-10240-7>
- [6] P. Viotti and M. Vukolić, “Consistency in Non-Transactional Distributed Storage Systems,” *ACM Computing Surveys*, vol. 49, no. 1, pp. 1–34, 2016, art. no. 19. doi: 10.1145/2926965
- [7] D. Narváez, N. Battaglia, A. Fernández, and G. Rossi, “Designing Microservices Using AI: A Systematic Literature Review,” *Software*, vol. 4, no. 1, art. no. 6, 2025. doi: <https://doi.org/10.3390/software4010006>
- [8] R. Laigner, Y. Zhou, M. A. Vaz Salles, Y. Liu, and M. Kalinowski, “Data Management in Microservices: State of the Practice, Challenges, and Research Directions,” *Proceedings of the VLDB Endowment*, vol. 14, no. 13, pp. 3348–3361, 2021. doi: <https://doi.org/10.14778/3484224.3484232>